

ScholarFlow — Demo Video Speech

Team Phantom Devs · Total duration: **10:00** (Introduction 1:35 · Core Features Demo 6:00 · Testing Demo 1:50 · GitHub & Close 0:35)

Introduction (0:00 – 1:35)

Opening (0:00 – 0:20)

Every researcher knows this feeling. Papers live in one app. Notes in another. Citations somewhere else. And the AI knows nothing about your actual work. This is the problem ScholarFlow solves.

What ScholarFlow Is (0:20 – 0:45)

Our team is Phantom Devs, and ScholarFlow is one place for the full research workflow. Upload a paper, understand it with AI, annotate it, write with it, cite it, and collaborate on it in real time. No existing tool combines all of these. ScholarFlow is the first.

Built to Be Trusted (0:45 – 1:20)

ScholarFlow is more than a paper library. It is the reading tool, the writing tool, the citation manager, the AI assistant, and the shared workspace — working together in one product. And because research must be trusted, correctness, permissions, and payment safety are engineered in from the start.

Setup (1:20 – 1:35)

I am logged in as Bob, a team lead. My library already has papers, and my team is active. Let me start with the most common task: adding a new paper.

Core Features Demo (1:35 – 7:35)

1. Upload and Automatic Metadata (1:35 – 2:15)

This is the upload page. I drop a research paper here. Watch the metadata panel. The title, the authors, and the abstract appear on their own — extracted by AI from the document. I

change nothing. I click publish. The paper is now in my library, stored in secure cloud storage.

Let me find it the way a researcher would — by meaning.

2. Library and Semantic Search (2:15 – 2:50)

This is my library — every paper, with tags, status, and collections. Now I search by meaning, not by exact words. I type "language models hallucinate", and the system finds the relevant paper — even though those words are not in its title. That is semantic search, powered by vector embeddings.

3. Reading, Highlighting, and Notes (2:50 – 3:25)

Here is the paper. The extracted text is ready, and the PDF preview is right here. I select a sentence and highlight it. I add a short note. I can also keep my own research notes beside the paper. Everything I mark stays attached to this paper — for me, and for my team.

4. AI Summary, Key Points, and Questions (3:25 – 4:20)

Now the AI layer. I click generate summary. In a few seconds I get the key findings — in the tone and length I chose. Below it, the key points: the main claims of the paper, extracted one by one. And I can ask questions. For example: which method did they use? The answer is grounded in this paper — not in the open internet.

5. Writing: Template, Citation, Export (4:20 – 5:15)

Next, writing. I create a new paper from a template — this one is IEEE. The editor supports tables, images, and LaTeX math, and it saves automatically — watch the save indicator. I insert a citation from my library. It is added in the format I choose — one of nine formats, including APA, IEEE, and BibTeX. Finally, export. One click, and the paper becomes a real PDF, ready to share.

6. Real-Time Co-Editing (5:15 – 6:15)

This is the part I am most proud of — two people, one paper, at the same time. On the left is my screen. On the right, my teammate Emily, in another browser. When she types, I see it instantly. Her cursor carries her name; mine carries mine. This is real-time co-editing, built directly into the research workflow.

And the same live system runs everything around the paper.

7. Team Invite and Live Notification (6:15 – 6:50)

Collaboration also means management. I invite a new member and choose a role — Viewer, Editor, or Manager. The moment I send it, the notification arrives in the bell, in real time. Invitations, role changes, and removals all flow through the same live channel.

8. Payments and Admin Proof (6:50 – 7:35)

Finally, the business side. I upgrade to the Pro plan. This is Stripe Checkout — hosted by Stripe, so no card data touches our servers. I use a test card. The payment succeeds. The billing page shows my plan and my invoices. And in the admin console, the same payment appears — because our backend processes Stripe webhooks.

Testing Demo (7:35 – 9:25)

Why Testing Matters (7:35 – 7:45)

That is the product. Now — how do we know it is correct? We test in three layers: automated gates, live system health, and security.

1. Quality Gates (7:45 – 8:05)

Every change must pass three gates: the production build, the type check, and the linter. Here, the type check passes for all three applications, and the linter reports zero errors. This runs before anything ships.

2. Health and Query Budget (8:05 – 8:25)

This is the health endpoint — database, memory, and services, all green. The backend log tracks every slow database query. Our budget is fifty milliseconds, and nothing exceeds it.

3. Access Control (8:25 – 9:05)

Now security. Three quick checks. One: a view-only colleague tries to edit. The server refuses — four oh three. The server decides, never the interface. Two: a draft paper has no public link, and opening it gives a clean not-found page — no data leaks. Three: when I revoke a share, access disappears immediately.

4. Payment Safety (9:05 – 9:25)

For payments, webhooks are the source of truth. I replay the same event twice — the subscription changes once. Handlers are idempotent, and a sweeper handles failed payments after a grace period.

GitHub & Engineering Practice (9:25 – 10:00)

Built with Industry Practice (9:25 – 9:55)

Before we finish, one last look — our GitHub. Sprint-based development: every feature in its own branch and pull request, with contributions from every member. Every version was released properly, and the product is deployed live.

Closing (9:55 – 10:00)

That is ScholarFlow — built by a team, released step by step, and running today. Thank you for watching.